

# **Software Requirements Specification**

for

## **Academic Integrity Submission & Similarity Analysis System**

**Version:** 1.0

**Prepared by:** Group 16 — Muhammad Nabeel Waheed, Thomas Neal, Ghassan Balouze,  
Kev Akpinar, Abdel Zahran, Ty Mabee

**Kool Group Name:** Last Minute Warriors

**Course:** COSC 4P02 — Course Project

**Date:** 2026-02-10

## Contents

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>3</b> |
| 1.1      | Document Purpose . . . . .                                       | 3        |
| 1.2      | Document Conventions . . . . .                                   | 3        |
| 1.3      | Project Scope . . . . .                                          | 3        |
| 1.4      | References . . . . .                                             | 3        |
| 1.5      | Intended Use of GitHub . . . . .                                 | 3        |
| <b>2</b> | <b>Overall Description</b>                                       | <b>4</b> |
| 2.1      | Product Perspective . . . . .                                    | 4        |
| 2.2      | User Classes and Characteristics . . . . .                       | 5        |
| 2.3      | Operating Environment . . . . .                                  | 5        |
| 2.4      | Design and Implementation Constraints . . . . .                  | 6        |
| 2.5      | Assumptions and Dependencies . . . . .                           | 6        |
| <b>3</b> | <b>System Features</b>                                           | <b>7</b> |
| 3.1      | User Authentication and Role-Based Access Control . . . . .      | 7        |
| 3.1.1    | Description . . . . .                                            | 7        |
| 3.1.2    | Stimulus/Response Sequences . . . . .                            | 7        |
| 3.1.3    | Functional Requirements . . . . .                                | 7        |
| 3.2      | Offering and Assignment Management . . . . .                     | 7        |
| 3.2.1    | Description . . . . .                                            | 7        |
| 3.2.2    | Stimulus/Response Sequences . . . . .                            | 7        |
| 3.2.3    | Functional Requirements . . . . .                                | 7        |
| 3.3      | Submission Upload and Storage . . . . .                          | 8        |
| 3.3.1    | Description . . . . .                                            | 8        |
| 3.3.2    | Stimulus/Response Sequences . . . . .                            | 8        |
| 3.3.3    | Functional Requirements . . . . .                                | 8        |
| 3.4      | Template Management and Mandatory Template Subtraction . . . . . | 9        |
| 3.4.1    | Description . . . . .                                            | 9        |
| 3.4.2    | Stimulus/Response Sequences . . . . .                            | 9        |
| 3.4.3    | Functional Requirements . . . . .                                | 9        |
| 3.5      | Repository Selection for Cross-Offering Comparison . . . . .     | 9        |
| 3.5.1    | Description . . . . .                                            | 9        |
| 3.5.2    | Stimulus/Response Sequences . . . . .                            | 9        |
| 3.5.3    | Functional Requirements . . . . .                                | 10       |
| 3.6      | Similarity Analysis Jobs . . . . .                               | 10       |
| 3.6.1    | Description . . . . .                                            | 10       |
| 3.6.2    | Stimulus/Response Sequences . . . . .                            | 10       |
| 3.6.3    | Functional Requirements . . . . .                                | 12       |
| 3.7      | Instructor Reporting and Comparison View . . . . .               | 12       |
| 3.7.1    | Description . . . . .                                            | 12       |
| 3.7.2    | Stimulus/Response Sequences . . . . .                            | 12       |
| 3.7.3    | Functional Requirements . . . . .                                | 12       |
| 3.8      | Anonymization and Privacy Controls . . . . .                     | 12       |
| 3.8.1    | Description . . . . .                                            | 12       |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 3.8.2    | Stimulus/Response Sequences . . . . .                                    | 13        |
| 3.8.3    | Functional Requirements . . . . .                                        | 13        |
| <b>4</b> | <b>Data Requirements</b>                                                 | <b>13</b> |
| 4.1      | Logical Data Model . . . . .                                             | 13        |
| 4.2      | Data Dictionary . . . . .                                                | 13        |
| <b>5</b> | <b>External Interface Requirements</b>                                   | <b>14</b> |
| 5.1      | User Interfaces . . . . .                                                | 14        |
| 5.2      | Software Interfaces . . . . .                                            | 14        |
| 5.3      | Communications Interfaces . . . . .                                      | 14        |
| <b>6</b> | <b>Quality Attributes</b>                                                | <b>14</b> |
| 6.1      | Usability . . . . .                                                      | 14        |
| 6.2      | Performance . . . . .                                                    | 15        |
| 6.3      | Security . . . . .                                                       | 15        |
| <b>7</b> | <b>Use Cases</b>                                                         | <b>15</b> |
| 7.1      | UC-01: Student Submits an Assignment . . . . .                           | 15        |
| 7.1.1    | Main Success Scenario . . . . .                                          | 15        |
| 7.1.2    | Alternate / Exception Flows . . . . .                                    | 16        |
| 7.2      | UC-02: Instructor Uploads Template (Starter Code) . . . . .              | 17        |
| 7.2.1    | Main Success Scenario . . . . .                                          | 17        |
| 7.2.2    | Alternate / Exception Flows . . . . .                                    | 17        |
| 7.3      | UC-03: Instructor Runs Similarity Analysis . . . . .                     | 18        |
| 7.3.1    | Main Success Scenario . . . . .                                          | 19        |
| 7.3.2    | Alternate / Exception Flows . . . . .                                    | 19        |
| 7.4      | UC-04: Instructor Reviews Results and Compares Two Submissions . . . . . | 19        |
| 7.4.1    | Main Success Scenario . . . . .                                          | 19        |
| 7.4.2    | Alternate / Exception Flows . . . . .                                    | 19        |
| <b>8</b> | <b>UML and Design Documentation</b>                                      | <b>20</b> |
| 8.1      | Key Architectural Decisions . . . . .                                    | 20        |
| 8.2      | Diagram Index (Files) . . . . .                                          | 20        |
| <b>9</b> | <b>Glossary</b>                                                          | <b>20</b> |

## 1 Introduction

### 1.1 Document Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Academic Integrity Submission & Similarity Analysis System. The intended audience includes instructors (stakeholders), developers, testers, and maintainers. The document describes the system behavior, interfaces, constraints, and quality attributes needed to implement and validate the system.

### 1.2 Document Conventions

Requirements are labeled using the following identifiers:

- **FR-**: Functional Requirement
- **NFR-**: Non-Functional Requirement
- **DR-**: Data Requirement
- **UC-**: Use Case

**Shall** indicates a mandatory requirement; **should** indicates a strong recommendation.

### 1.3 Project Scope

The system provides a web-based platform where students submit programming assignments, and instructors review similarity analysis results to support academic integrity investigations. Submissions are compared against other submissions in the same course offering and optionally against one or more secondary repositories (e.g., previous offerings or instructor-defined repositories). The system must anonymize submissions and results to protect personal information and comply with privacy constraints. Minimum language support includes C, C++, and Java. The comparison engine must be implemented by the team and cannot be delegated to a third-party AI service.

### 1.4 References

- Project Statement (provided by instructor)
- Phase 1 Marking / Requirements Notes (course document)
- Software Requirements Specification Template

### 1.5 Intended Use of GitHub

Project coordination, issue tracking, milestones, documentation, and version control shall be maintained in a shared GitHub repository. This includes keeping UML source project files (e.g., diagrams) under version control.

## 2 Overall Description

### 2.1 Product Perspective

The product is a web application consisting of a browser-based client (student and instructor views), a backend API server, persistent storage for submissions and metadata, and a background analysis worker (engine) that computes similarity results. The system may be deployed on a server or in containers. Clients access it over HTTPS via a standard web browser.

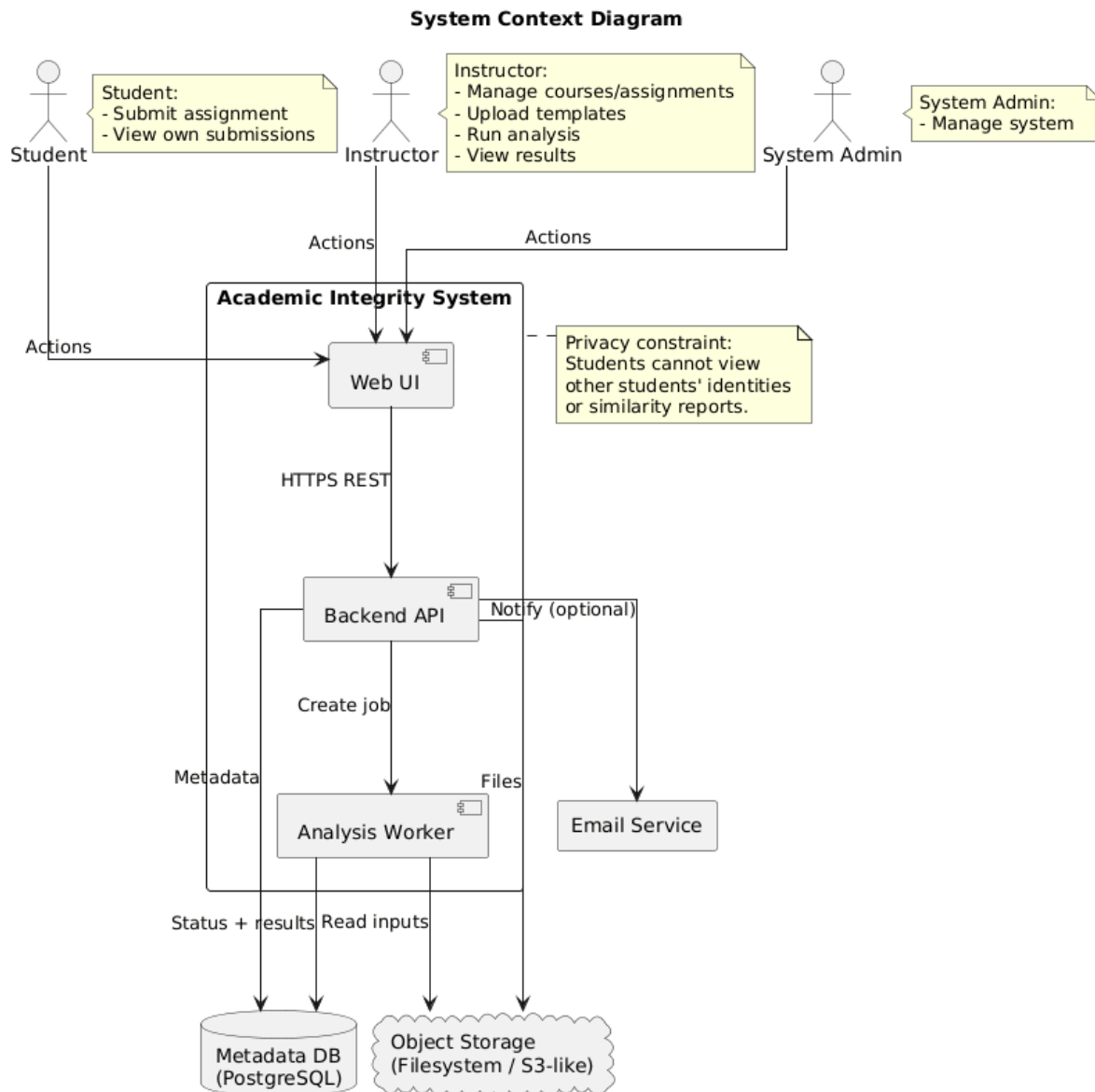


Figure 1: System Context Diagram

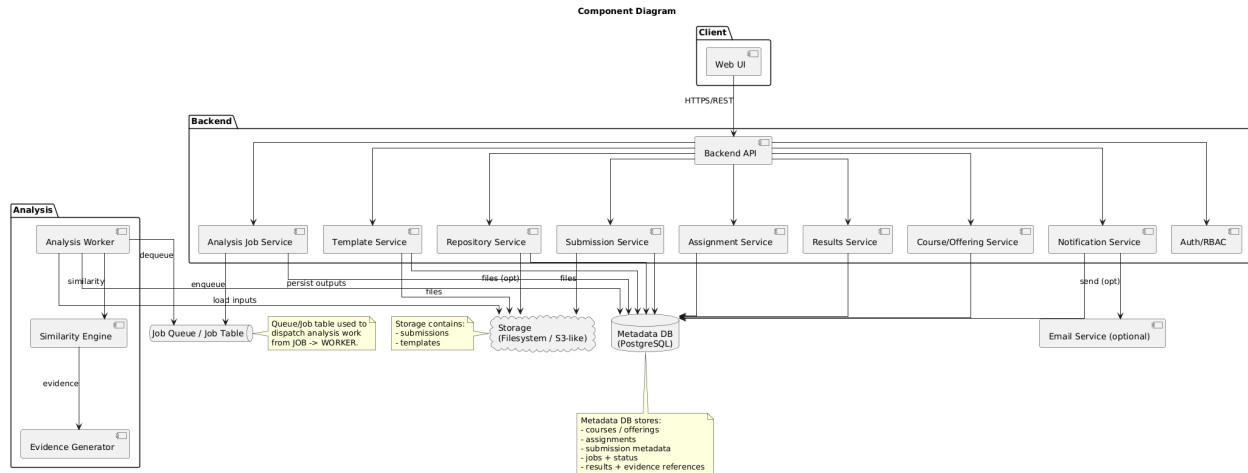


Figure 2: Component Diagram

## 2.2 User Classes and Characteristics

- **Student:** uploads submissions and views their own submission history/status.
- **Instructor:** configures assignments/repositories/templates, triggers analysis, reviews results, compares submissions.
- **System Administrator:** manages deployment configuration, backups, and operational monitoring.

## 2.3 Operating Environment

- **Client:** modern web browsers (Chrome/Firefox/Safari/Edge).
- **Server:** Linux-based host or container platform (e.g., Docker), running a web server and database.
- **Storage:** local filesystem or S3-like object storage; relational database for metadata.
- **Network:** HTTPS over the public Internet.

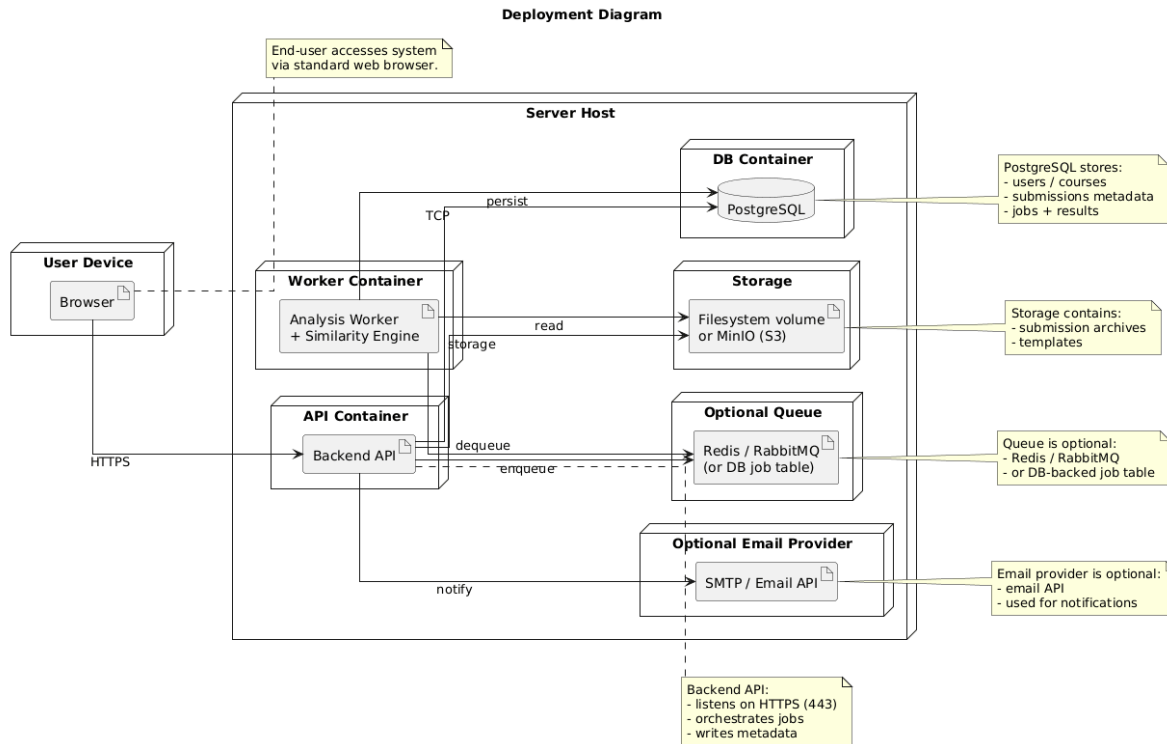


Figure 3: Deployment Diagram

## 2.4 Design and Implementation Constraints

- **NFR-001** The code-comparison function shall not be outsourced to an external AI service.
- **NFR-002** The system shall support C, C++, and Java at minimum.
- **NFR-003** The system shall protect personal information and anonymize outputs for non-instructor users.
- **NFR-004** The system shall use HTTPS for all client-server communications in non-local deployments.
- **FR-019** The system shall support selecting multiple secondary repositories for comparison.
- **FR-014** Template (starter code) subtraction shall be applied prior to similarity scoring.

## 2.5 Assumptions and Dependencies

- Users have stable Internet access and a supported browser.
- Instructors provide assignment templates (starter code) in an accepted format (e.g., zip).
- Similarity accuracy depends on correct language detection and tokenization/parsing.

### 3 System Features

#### 3.1 User Authentication and Role-Based Access Control

##### 3.1.1 Description

**Priority: High.** Supports user login and enforces permissions for Students and Instructors; prevents disclosure of other students' identities and submissions.

##### 3.1.2 Stimulus/Response Sequences

- Student logs in → system authenticates → student dashboard is displayed.
- Instructor logs in → system authenticates → instructor dashboard is displayed.
- Unauthorized user requests restricted resource → system denies access and logs event.

##### 3.1.3 Functional Requirements

1. **FR-001** The system shall authenticate users before allowing access to protected functionality.
2. **FR-002** The system shall support at minimum two roles: Student and Instructor.
3. **FR-003** The system shall enforce role-based access control (RBAC) for all API endpoints.
4. **FR-004** The system shall prevent Students from viewing other students' submissions, identities, or similarity reports.
5. **FR-005** The system shall allow Instructors to view submissions and similarity reports for offerings they manage.

#### 3.2 Offering and Assignment Management

##### 3.2.1 Description

**Priority: High.** Allows instructors to create course offerings and assignments, configure due dates, and define comparison settings per assignment.

##### 3.2.2 Stimulus/Response Sequences

- Instructor creates an offering → system stores offering metadata → offering appears in instructor dashboard.
- Instructor creates an assignment → system stores assignment → assignment appears to enrolled students.

##### 3.2.3 Functional Requirements

1. **FR-006** The system shall allow Instructors to create, edit, and archive course offerings.
2. **FR-007** The system shall allow Instructors to create and edit assignments within an offering.
3. **FR-008** The system shall store assignment metadata including title, due date, allowed languages, and submission rules.



### 3.3 Submission Upload and Storage

#### 3.3.1 Description

**Priority: High.** Allows students to submit code files for an assignment and stores submissions in durable storage with metadata for later analysis.

#### 3.3.2 Stimulus/Response Sequences

- Student uploads a submission → system validates file type/size → stores submission → returns submission ID and timestamp.
- Student views submission status → system returns submission metadata.

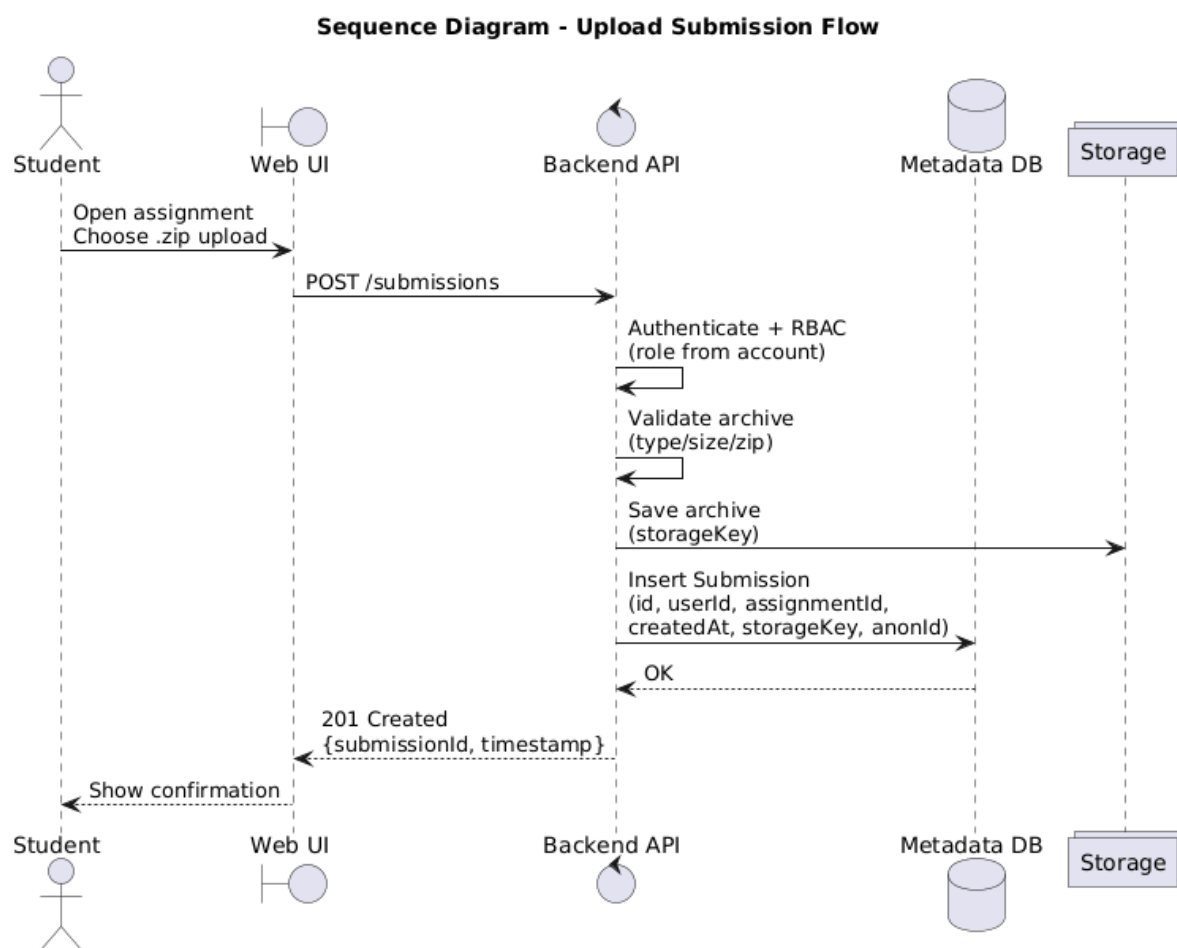


Figure 4: Sequence Diagram: Upload Submission Flow

#### 3.3.3 Functional Requirements

1. **FR-009** The system shall accept student submissions via a web-based upload flow.
2. **FR-010** The system shall support submissions containing multiple files (e.g., zip archive) and/or single-file uploads.

3. **FR-011** The system shall store submissions in server-side storage and maintain metadata in the database.
4. **FR-012** The system shall compute and store a submission ID for each stored submission.
5. **FR-013** The system shall retain historical submissions per instructor policy or course rules.

### 3.4 Template Management and Mandatory Template Subtraction

#### 3.4.1 Description

**Priority: High.** Allows instructors to upload starter code templates and ensures template subtraction is applied before similarity scoring.

#### 3.4.2 Stimulus/Response Sequences

- Instructor uploads template → system stores template version → builds template fingerprints → marks template active for assignment.
- Analysis runs → system subtracts template artifacts from each submission before computing similarity.

#### 3.4.3 Functional Requirements

1. **FR-014** The system shall allow Instructors to upload one or more template files for each assignment.
2. **FR-015** The system shall version templates and retain previous versions for auditability.
3. **FR-016** The system shall build and store template fingerprints/representations used for subtraction.
4. **FR-017** The system shall subtract template-derived artifacts from submission artifacts before similarity scoring.
5. **FR-018** The system shall record which template version was used for each analysis job.

### 3.5 Repository Selection for Cross-Offering Comparison

#### 3.5.1 Description

**Priority: Medium.** Supports configuring multiple comparison repositories per assignment (current offering and optional secondary repositories).

#### 3.5.2 Stimulus/Response Sequences

- Instructor enables one or more repositories for an assignment → system saves configuration → future analyses include those repositories.

### 3.5.3 Functional Requirements

1. **FR-019** The system shall support multiple secondary repositories that can be selected for comparison by the Instructor.
2. **FR-020** The system shall compare submissions against (a) same-offering submissions and (b) enabled secondary repositories.
3. **FR-021** The system shall allow Instructors to enable/disable repositories per assignment without redeploying the system.

## 3.6 Similarity Analysis Jobs

### 3.6.1 Description

**Priority: High.** Computes similarity indices between submissions and stores results. Analysis executes asynchronously to avoid blocking uploads.

### 3.6.2 Stimulus/Response Sequences

- Instructor triggers analysis → system queues a job → worker processes comparisons → job marked complete → results become available.
- Instructor checks status → system returns job state and progress information.

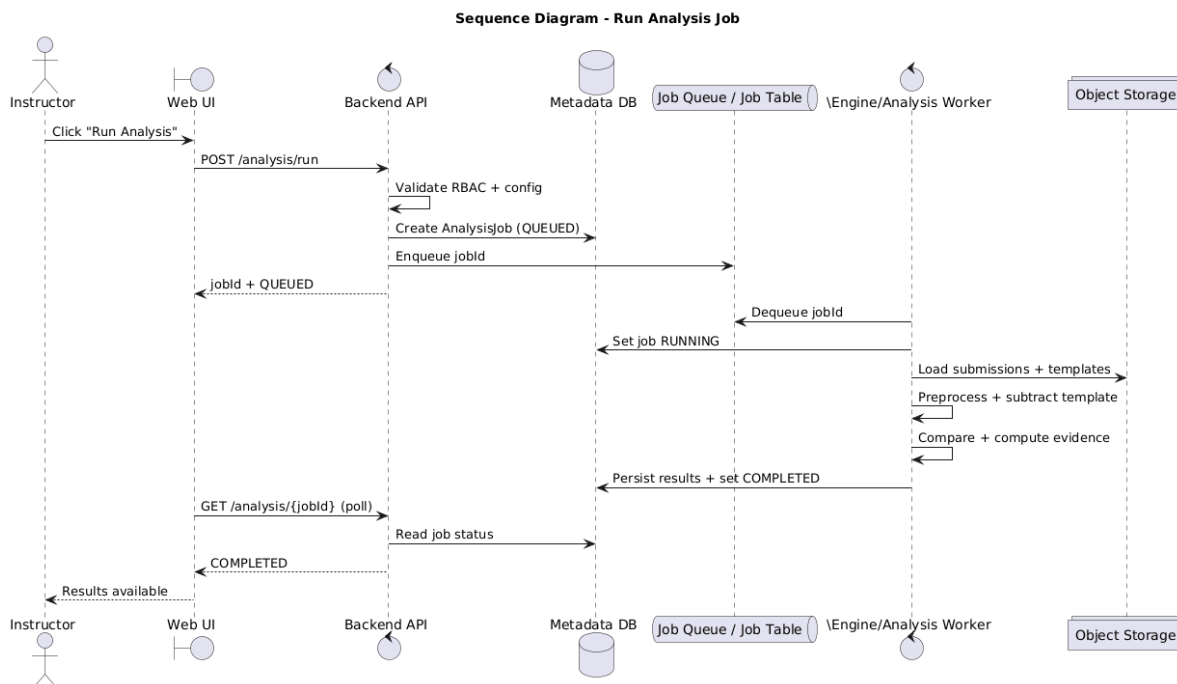


Figure 5: Sequence Diagram: Run Similarity Analysis Job

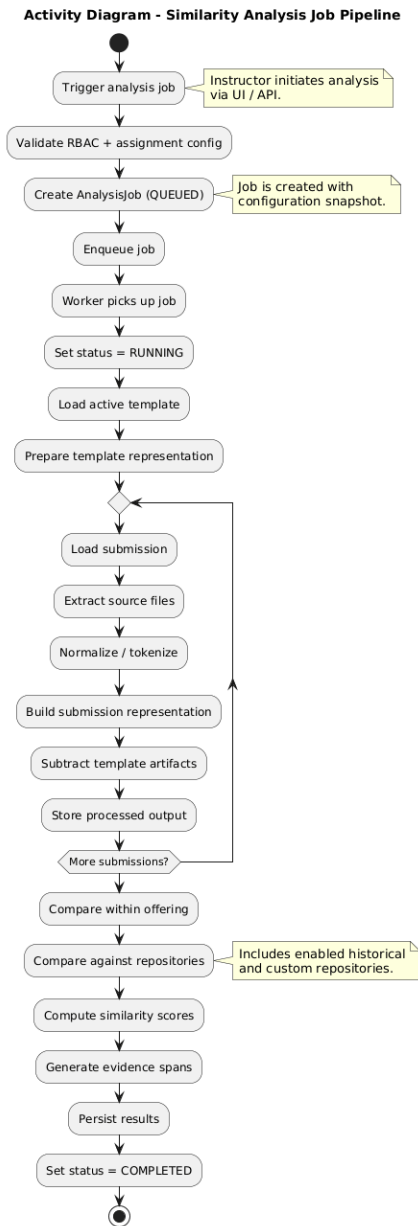


Figure 6: Activity Diagram: Similarity Analysis Pipeline

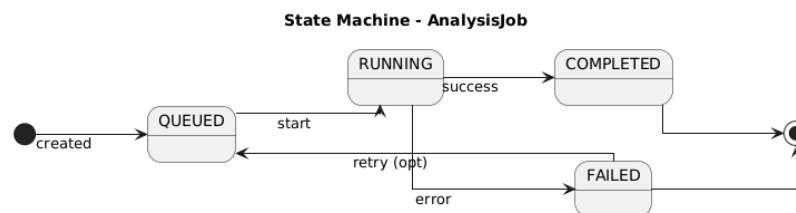


Figure 7: State Machine Diagram: AnalysisJob Lifecycle

### 3.6.3 Functional Requirements

1. **FR-022** The system shall allow Instructors to trigger similarity analysis for an assignment.
2. **FR-023** The system shall execute analysis asynchronously (background worker).
3. **FR-024** The system shall compute similarity scores for compared pairs and store results with timestamps.
4. **FR-025** The system shall support token-based similarity at minimum; additional methods may be used as enhancements.
5. **FR-026** The system shall record the repositories included in each analysis run.
6. **FR-027** The system shall provide job status: QUEUED, RUNNING, COMPLETED, FAILED.

## 3.7 Instructor Reporting and Comparison View

### 3.7.1 Description

**Priority: High.** Provides a web UI for instructors to review similarity results and compare two submissions in a user-friendly manner.

### 3.7.2 Stimulus/Response Sequences

- Instructor opens results → system shows ranked matches with scores and repository sources.
- Instructor selects a pair → system shows side-by-side code with highlighted matching spans (when available).

### 3.7.3 Functional Requirements

1. **FR-028** The system shall present similarity results in a sortable/filterable table (e.g., by score threshold).
2. **FR-029** The system shall allow an Instructor to open a comparison view for any matched pair.
3. **FR-030** The comparison view shall show side-by-side code with evidence highlights when evidence is available.
4. **FR-031** The system shall allow exporting reports (CSV/PDF) containing summary match information.
5. **FR-032** The system shall log report access events for audit purposes.

## 3.8 Anonymization and Privacy Controls

### 3.8.1 Description

**Priority: High.** Ensures personal information is protected and only authorized users can access identifying details.

### 3.8.2 Stimulus/Response Sequences

- Student requests analysis results → system returns only their own submission metadata (no pair results).
- Non-instructor client requests submission → system returns anonymized identifiers unless instructor authorization is present.

### 3.8.3 Functional Requirements

1. **FR-033** The system shall assign anonymized identifiers (pseudonyms) to submissions for use in reports.
2. **FR-034** The system shall restrict identity resolution (mapping pseudonym to student identity) to authorized Instructors.
3. **FR-035** The system shall ensure all student-facing APIs return only the requesting student's information.
4. **FR-036** The system shall implement access logging for sensitive operations (e.g., viewing reports, downloading submissions).

## 4 Data Requirements

### 4.1 Logical Data Model

Core entities include: User, Offering, Assignment, Submission, Template, Repository, AnalysisJob, SimilarityPair, and EvidenceSpan. Offerings contain Assignments; Assignments have Submissions and Templates; AnalysisJobs produce SimilarityPairs; SimilarityPairs reference two Submissions and a Repository source; EvidenceSpans map similarity evidence to file/line ranges.

### 4.2 Data Dictionary

| Entity      | Key Fields                                                                                        | Description                                               | Notes                           |
|-------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------|---------------------------------|
| Submission  | id, assignment_id, user_id, anon_id, created_at, language, storage_key, sha256                    | A single student upload for an assignment.                | anon_id used in reports.        |
| Template    | id, assignment_id, version, storage_key, created_at                                               | Starter code provided by instructor.                      | Used for mandatory subtraction. |
| Repository  | id, name, type, created_by                                                                        | Comparison source (current offering, historical, custom). | Enabled per assignment.         |
| AnalysisJob | id, assignment_id, status, created_at, finished_at, triggered_by, template_version, repo_snapshot | Async analysis execution record.                          | Stores config snapshot.         |

| Entity         | Key Fields                                                                            | Description                   | Notes                                    |
|----------------|---------------------------------------------------------------------------------------|-------------------------------|------------------------------------------|
| SimilarityPair | id, job_id, sub_a_id, sub_b_id, repo_id, score_primary, score_secondary, evidence_ref | Similarity result for a pair. | evidence_ref points to spans JSON/table. |

## 5 External Interface Requirements

### 5.1 User Interfaces

- Student Dashboard: list assignments, upload submission, view own submission timestamps/status.
- Instructor Dashboard: manage offerings/assignments, upload templates, select repositories, run analysis, view job status.
- Results View: filter/sort similarity pairs, open pair details.
- Pair Comparison View: side-by-side code with evidence highlighting.

### 5.2 Software Interfaces

- Database: relational database storing metadata (users, offerings, submissions, jobs, results).
- Storage: filesystem or object storage for uploaded archives and extracted files.
- Authentication: application login.

### 5.3 Communications Interfaces

- The system shall use HTTPS for all client-server communications. (**NFR-004**)
- The system shall support file uploads via standard HTTP multipart transfer.

## 6 Quality Attributes

This section defines measurable non-functional requirements (NFRs) for usability, performance, and security. Each NFR includes a brief rationale and a suggested verification approach (test/inspection).

### 6.1 Usability

- **NFR-005 Results table filtering/sorting.** The instructor results table shall support filtering by similarity score threshold and sorting by score.
- **NFR-006 Comparison view clarity.** The comparison view shall provide clear highlighting and navigation between matching regions.
- **NFR-007 Submission flow simplicity.** The student submission flow shall complete in three steps or fewer (Select → Upload → Confirm).
- **NFR-008 Safe error messaging.** Errors shall be actionable and shall not expose sensitive internal details.

## 6.2 Performance

- **NFR-010 Upload responsiveness.** The system shall accept and persist an upload within 10 seconds for files up to the configured size limit under normal load.
- **NFR-011 Analysis throughput.** For a class of 200 submissions, the system should complete a same-offering analysis run within an hour on baseline hardware.
- **NFR-012 Asynchronous trigger.** Triggering analysis should return a job ID within 2 seconds under normal load.
- **NFR-013 Storage efficiency.** The system should avoid unnecessary duplication of stored/extracted files across repeated runs.

## 6.3 Security

- **NFR-014 RBAC (deny by default).** The system shall enforce RBAC on all endpoints.
- **NFR-015 Audit logging.** The system shall log access to reports and submission downloads.
- **NFR-016 Upload validation.** The system shall validate uploads to prevent malicious file types and zip-slip traversal.
- **NFR-017 Secret handling.** The system shall protect secrets/configuration (env vars/secrets store) and avoid hard-coding in source control.
- **NFR-018 Privacy/anonymization.** Student-facing endpoints shall never expose other students' identities, submissions, or similarity results; instructor results shall use anonymized identifiers by default.

## 7 Use Cases

### 7.1 UC-01: Student Submits an Assignment

**Primary Actor(s):** Student

**Preconditions:** Student is authenticated and is enrolled in the offering; assignment exists and accepts submissions.

**Trigger:** Student selects an assignment and chooses files to upload.

#### 7.1.1 Main Success Scenario

1. Student navigates to the assignment page.
2. Student selects a single file or a zip archive containing source files.
3. System validates the upload (size, type, zip structure) and rejects unsafe content.
4. System stores the uploaded archive and records submission metadata (timestamp, language, hash).
5. System displays a confirmation with the submission ID and timestamp.



### 7.1.2 Alternate / Exception Flows

- Upload rejected: file too large or unsupported type → system returns a validation error.
- Upload rejected: zip-slip/path traversal detected → system rejects and logs the event.
- Student not authorized → system denies access.

**Postconditions:** Submission is persisted in storage; metadata is stored and visible to the student in their submission history.

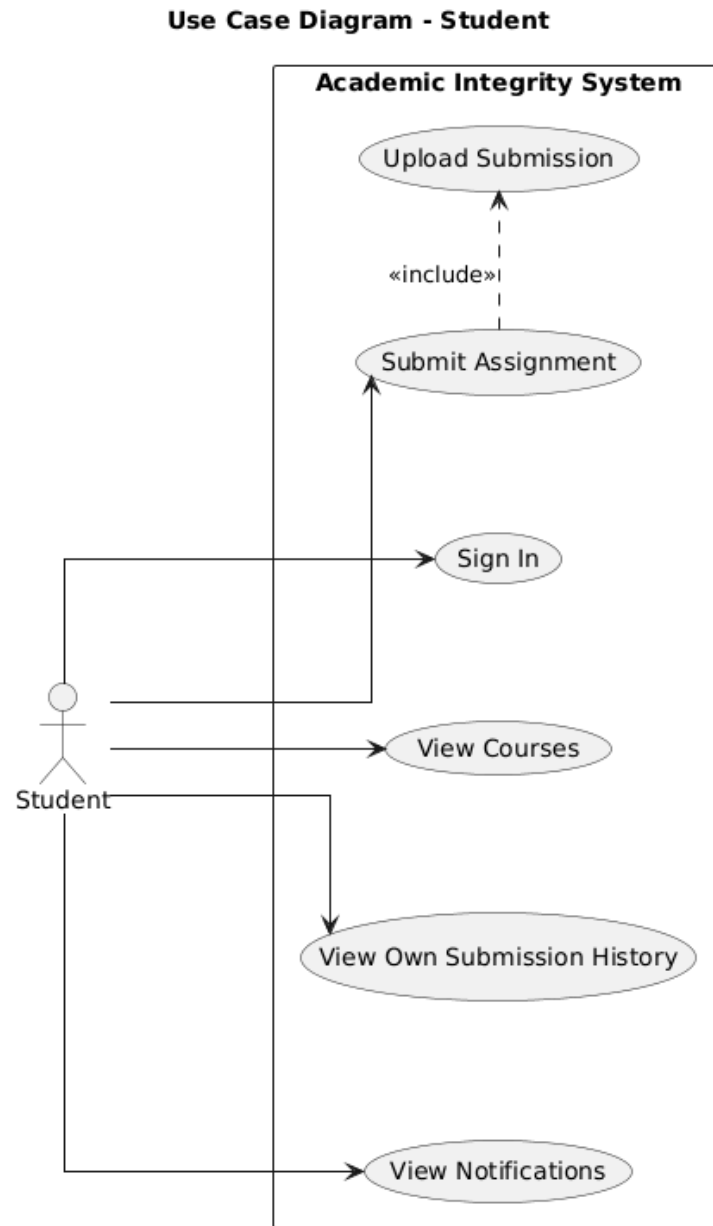


Figure 8: Use Case Diagram: Student

## 7.2 UC-02: Instructor Uploads Template (Starter Code)

**Primary Actor(s):** Instructor

**Preconditions:** Instructor is authenticated and owns/manages the offering; assignment exists.

**Trigger:** Instructor uploads a template package for the assignment.

### 7.2.1 Main Success Scenario

1. Instructor navigates to assignment settings.
2. Instructor uploads a template archive (zip) and optional notes (e.g., which files are starter).
3. System validates the archive and stores it as a new template version.
4. System builds template fingerprints/representations and marks the template version as active for the assignment.
5. System displays the active template version and build status.

### 7.2.2 Alternate / Exception Flows

- Template rejected: invalid archive structure → system returns validation error.
- Fingerprint build failed → system marks template as FAILED and provides an error message to instructor.

**Postconditions:** An active template version exists for the assignment and will be used for mandatory template subtraction.

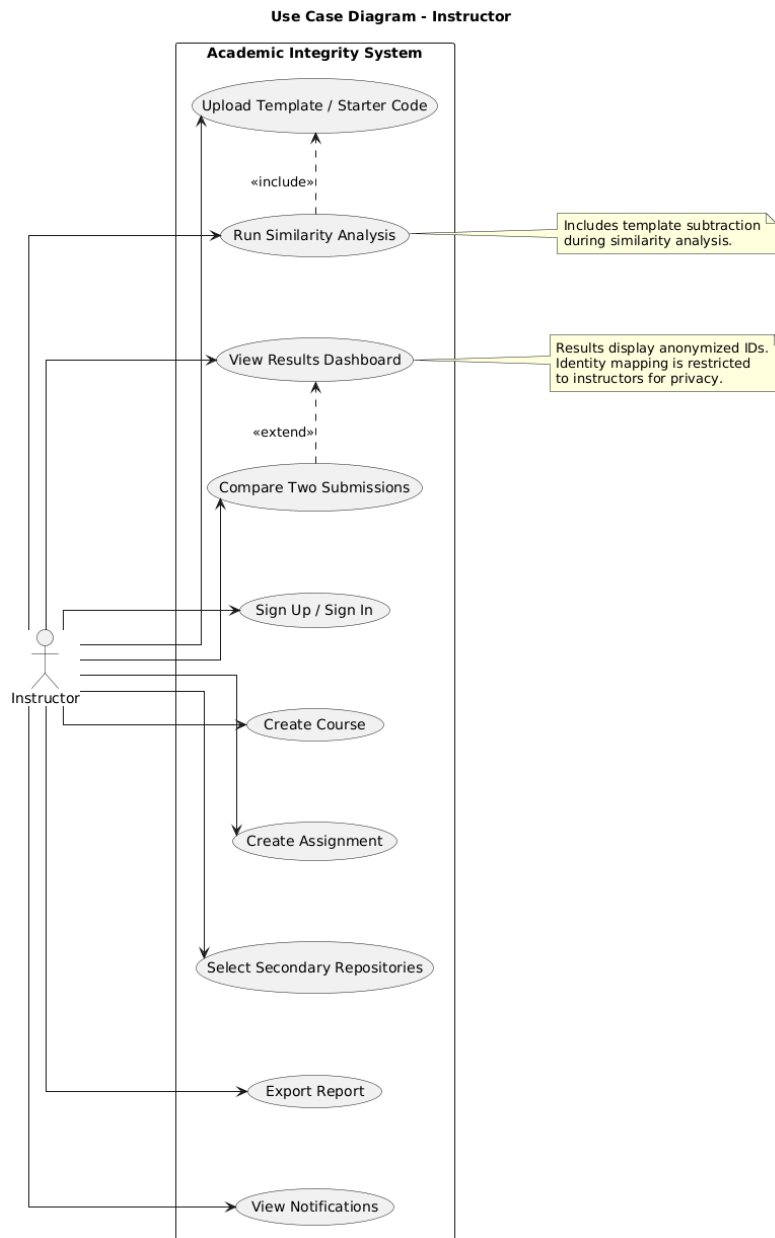


Figure 9: Use Case Diagram: Instructor

### 7.3 UC-03: Instructor Runs Similarity Analysis

**Primary Actor(s):** Instructor

**Preconditions:** Instructor authenticated; assignment has submissions; an active template is available; selected repositories configured.

**Trigger:** Instructor clicks “Run Analysis” for an assignment.

### 7.3.1 Main Success Scenario

1. System creates an AnalysisJob record with a snapshot of settings (template version, repositories, parameters).
2. System queues the job for the analysis worker and immediately returns job ID and status QUEUED.
3. Worker loads submissions and template representation; computes per-submission representations and applies template subtraction.
4. Worker compares submissions against same-offering submissions and enabled secondary repositories.
5. Worker stores similarity results and evidence pointers and marks job COMPLETED.

### 7.3.2 Alternate / Exception Flows

- Job fails due to internal error → system marks job FAILED, stores the error, and allows retry.
- No submissions available → system warns instructor and does not create a job.

**Postconditions:** Analysis results are stored and visible to the instructor; job status reflects final state.

## 7.4 UC-04: Instructor Reviews Results and Compares Two Submissions

**Primary Actor(s):** Instructor

**Preconditions:** An analysis job has completed successfully.

**Trigger:** Instructor opens the results table and selects a suspicious pair.

### 7.4.1 Main Success Scenario

1. System displays results with sorting/filtering controls (threshold, repository source).
2. Instructor selects a pair from the list.
3. System loads the two submissions and evidence spans (if available).
4. System renders a side-by-side comparison view and highlights matching spans.
5. Instructor optionally exports the report as CSV/PDF.

### 7.4.2 Alternate / Exception Flows

- Evidence not available (e.g., heavy refactor) → system still shows scores and labels the pair as “manual review recommended”.
- Download denied due to authorization error → system blocks and logs access attempt.

**Postconditions:** Instructor can record observations externally; system logs report access for auditability.

## 8 UML and Design Documentation

### 8.1 Key Architectural Decisions

- **ADR-001** Asynchronous analysis: similarity analysis runs in a worker to avoid blocking uploads.
- **ADR-002** Storage separation: submission binaries stored in filesystem/object storage; meta-data stored in relational DB.
- **ADR-003** Template subtraction: template representation is computed per assignment version and subtracted prior to scoring.

### 8.2 Diagram Index (Files)

All diagrams are included in the most relevant sections above:

- `system_context_diagram.png`
- `component_diagram.png`
- `deployment_diagram.png`
- `use_case_student.png`
- `use_case_instructor.png`
- `sequence_diagram_upload_flow.png`
- `sequence_diagram_Analysis.png`
- `activity_diagram.png`
- `state_machine.png`

## 9 Glossary

| Term                       | Definition                                                                                  |
|----------------------------|---------------------------------------------------------------------------------------------|
| Tokenization               | Breaking source code into tokens (keywords, identifiers, operators, literals).              |
| Normalization              | Transforming tokens to reduce superficial differences (e.g., renaming identifiers to ID).   |
| Fingerprint                | A compact set of hashed features representing a submission after pre-processing.            |
| Winnowing                  | A fingerprint-selection method that chooses representative hashes from sliding windows.     |
| GST (Greedy String Tiling) | Algorithm that finds non-overlapping matching token substrings (tiles) to provide evidence. |
| Template Subtraction       | Removing template-derived features from submissions before computing similarity scoring.    |

---

| <b>Term</b> | <b>Definition</b>                                                                        |
|-------------|------------------------------------------------------------------------------------------|
| Repository  | A collection of submissions used as a comparison source (current offering or secondary). |
| Offering    | A specific course instance/term (e.g., Winter 2026).                                     |
| RBAC        | Role-Based Access Control: enforcing permissions based on user role.                     |

---